

Unit Name: Getting Started with 'C'

Programming Languages:

"The languages which are used to communicate with the computer are called as programming languages i.e. with the help of programming languages, we are able to create or develop new program (i.e. software)"

Generally all programming languages are divided into two categories viz (Namely):

1) Problem oriented language (High Level Language):-

These languages are helpful for the programmer to develop or create the program very fastly but execution speed of such languages is very slow.

e.g. FORTRON, BASIC, PASCAL etc are the Problem oriented languages.

2) Machine oriented language (Low Level Language):-

These languages have been designed to give faster program execution but program development speed is very slow.

e.g. Machine or Assembly language.

Note:

'C' language is lies between these two categories i.e it supports for problem oriented language & also for machine oriented language therefore program developing speed and program execution speed is fast. And hence 'C' is Middle level language.

History of 'C' language (Evolution of C language):

Following table shows the evolution of 'C' language:

Name of programming language	Developed by	Drawback
ALGOL	International Committee In year 1960	It is too abstract and too general
CPL (Combined Programming language)	At Cambridge university and university of London in year 1963	Hard to learn & difficult to implement
BCPL(Basic Combined Programming language)	Martin Richards at Cambridge university in year 1967	It is less powerful and too specific.
B language	Ken Thompson at AT and T's Bell's laboratory in year 1970	It is machine dependant
C language	Dennis Ritchie at AT and T's Bells laboratory In USA in year 1972	It is not Object Oriented programming language.

In year 1960, there were some programming languages such as ALGOL, CPL, BCPL, B etc.

ALGOL is programming language which is root of all modern programming languages and which was developed by international committee in year 1960. This language was modular and structured but it was found to be too abstract and too general.

After ALGOL, **CPL** (Combined Programming Language) was developed at Cambridge University and university of London in year 1963. However it was hard to learn and difficult to implement.

After CPL, **BCPL** (Basic Combined Programming Language) was developed by Martin Richards at Cambridge University in year 1967. But it was less powerful and too specific.

After BCPL, **B** language was developed by Ken Thompson at AT and T's Bells laboratory in year 1970. But it was machine dependant language.

After B language, **C** language was developed by Dennis Ritchie at AT and T's Bell's laboratory in USA in year 1972.

Features / Characteristics / Advantages of 'C' language:

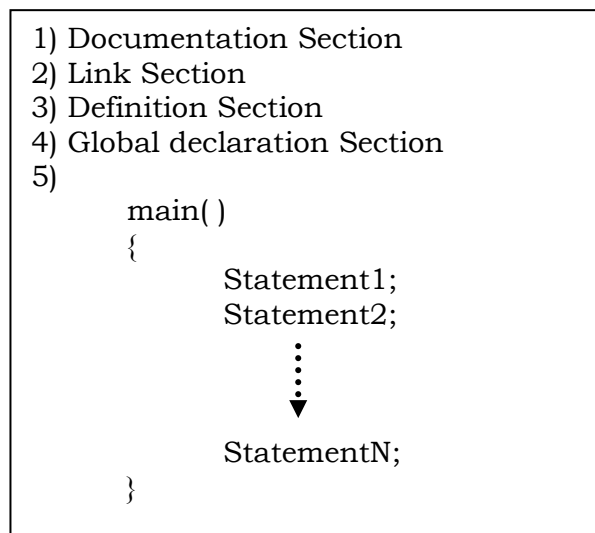
1) 'C' language is Robust Language:

'C' language consists of many inbuilt functions or strong operators which are used to write complex or strong program.

- 2) 'C' language is machine independent language (Portable language):
'C' language does not depend on configuration of machine that's why program written on one computer can easily run onto another computer.
- 3) 'C' compiler combines the capability of high level language and also low level language, therefore program developing speed as well as program execution speed is high or fast.
- 4) 'C' language has ability to extend itself:
This means that we can add our own function (user defined) in 'C' library.
- 5) 'C' program consist of many functions this allows programming task very easy & simple
- 6) 'C' is procedure (Structured) oriented language that contain lots of library functions which are used to develop large program.

Structure of 'C' program:

The 'C' program contains the several sections which are shown in following figure:



1) Documentation Section:

The documentation section contains the general information related with the program. That is, it contains name of program, name of programmer, date of program and other details.

We can give this information by two ways:

I) Single Line Comment:

If there is only one comment line then we use single line comment and this is achieved by // symbol.

e.g.

```
// program name= Addition
```

II) Multi Line Comment:

If there are several comment lines then we use multi line comment and this is achieved by

```
/* -----
   -----
   ----- */
```

e.g.

```
/* Program name: Addition
   Programmer: Dennis Ritchie
   Date: 23-Aug-1972    */
```

Note that: The documentation section is optional section. It is only helpful for understanding the general information of program.

2) Link Section:

The link section provides the instructions to the compiler to link the functions from 'C' library. This is achieved by pre-processor directive #include

Syntax to use #include :

#include<Headerfile name>

e.g.

If we want to use printf() or scanf() inbuilt functions then we must have to include stdio.h header file in following manner:

#include<stdio.h>

3) Definition Section:

In definition section we can define all the symbolic constants, this is achieved by using preprocessor directive #define

Syntax:

#define symbolicName value

e.g.

#define PI 3.14

4) Global declaration Section:

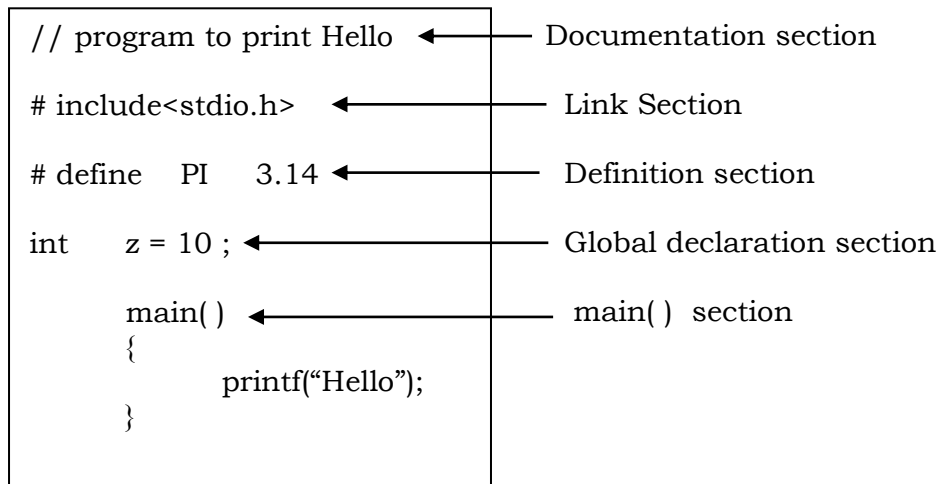
There are some variables that are used in more than one function then such variables are declared under global section and they are called as "Global variable"

Therefore global variables are easily accessed within every function of 'C' program.

5) main() function Section:

This is important and compulsory section because every 'C' program has one main() from which execution of program is starts. That is main() is starting point of 'C' program. Also the body of main() is defined by the user therefore it is user defined function.

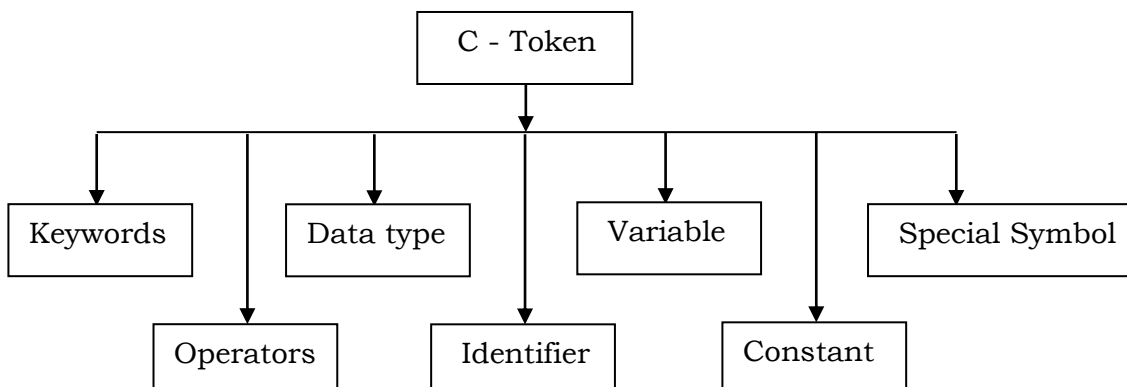
e.g.



'C' tokens:

"The 'C' token is the smallest individual unit or part of 'C' program."

Following tree diagram shows tokens available in 'C' language:



1) Keywords (Reserved Words):

“The words whose meaning is already known by the compiler are called as keywords or reserved words”

All keywords having fix meaning and we are not able to change that meaning.

All keywords in ‘C’ language are only in lower case letters.

There are 32 keywords present in ‘C’ library.

Following are the some keywords:

auto, static, extern, register, int, char, float, double, void, if, else, switch, case, break, continue, while, do, for, return, long, short, far, typedef, struct, union, enum etc.

2) Operators :

“Operator is a symbol that tells the computer to perform specific task”

OR

“Operator is a symbol which operates on the operands to perform specific task”

Types of Operators:

1) Arithmetic operators :

Arithmetic operators are used to perform arithmetic operations such as addition, subtraction, multiplication, division, modulation etc. Some arithmetic operators are shown in following table.

Operator	Meaning
+	Addition
—	Subtraction
*	Multiplication
/	Division
%	Modulation

e.g.

```
int    a, b, c;
      a = 10;
      b = 20;
      c = a + b;
```

Here, ‘c’ stores the addition of ‘a’ and ‘b’.

2) Relational operators :

Many times we have to compare two or more quantities with each other and take certain decision. Such type of comparison can be done by using relational operators. There are some relational operators which are shown in following table:

Operator	Meaning
<	Less than
>	Greater than
<=	Less than or equal to
>=	Greater than or equal to
!=	Not equal to
= =	Exactly equal to

e.g.

```
int    a;
      a = 5 ;
      if ( a > 0 )
          printf("Number is Positive");
      else
          printf("Number is Negative");
```

Here, out put: Number is positive

3) Logical operators :

Logical operators are used to perform logical operations for that 'C' language supports three logical operators and they are as follows:

Operator	Meaning
&&	Logical AND
	Logical OR
!	Logical NOT

e.g.

```

int    a;
a = 5 ;
if ( a > 0 && a == 5 )
    printf("Number is Positive and Five");
else
    printf("Number is Negative");

```

Explanation:

In above example, at a time two conditions are checked $a > 0$ and $a == 0$ both conditions are true therefore result is Number is Positive and Five. (Due to && operator both condition must be true then and then only block related with **if** is executed)

4) Assignment operator:

Assignment operator is used to assign or initialize value of an expression to the variable. This operator is denoted by symbol =

e.g.

```

int    x;
x = 10 ;

```

5) Increment and Decrement operator:

Increment Operator:

This operator is used to add value one into operands. This is unary operator i.e. it works on only one operand and it is denoted by symbol ++

e.g.

```

int    n=50;
Then, n++ gives answer 51
i.e. it works like    n = n + 1

```

Also we can use increment operator by two ways v.i.z. (namely) before the operand and after the operand therefore increment operator having two types:

I) Pre increment:

If we use increment operator before the operand then it is called as "pre increment" Which is shown in following example.

e.g.

```

int    x = 10 ;
printf("%d", ++x);    //10 value first incremented and then it assign to 'x'
printf("\n%d", x);

```

OUTPUT:

```

11
11

```

II) Post increment:

If we use increment operator after the operand then it is called as "post increment" Which is shown in following example.

e.g.

```

int    y = 50 ;
printf("%d", y++);    //50 value first assign to 'y' & then incremented
printf("\n%d", y);

```

OUTPUT:

```

50
51

```

Decrement Operator:

This operator is used to subtract value one from operands. This is unary operator i.e. it works on only one operand and it is denoted by symbol **--**

e.g. `int n=50;`

Then, `n --` gives answer 49

i.e. it works like `n = n - 1`

Also we can use decrement operator by two ways v.i.z. (namely) before the operand and after the operand therefore decrement operator having two types:

I) Pre decrement:

If we use decrement operator before the operand then it is called as “pre decrement” which is shown in following example.

e.g. `int x = 10 ;`
`printf(“%d”, -- x); //10 value first decremented and then it assign to ‘x’`
`printf(“\n%d”, x);`

OUTPUT:

9
9

II) Post decrement:

If we use decrement operator after the operand then it is called as “post decrement” which is shown in following example.

e.g. `int y = 50 ;`
`printf(“%d”, y -- ); //50 value first assign to ‘y’ & then decremented`
`printf(“\n%d”, y);`

OUTPUT:

50
49

6) Conditional operator: (Ternary Operator)

There are two conditional operators in ‘C’ language and they are denoted by symbol **?** and **:**

Syntax to Use conditional operator

`exp1 ? exp2 : exp3 ;`

Here, `exp1`, `exp2` and `exp3` are three expressions. And conditional operator is works as follow:

‘`exp1`’ is evaluated first, if it is True then ‘`exp2`’ is evaluated and value of ‘`exp2`’ becomes value of entire (complete) expression.

If value of ‘`exp1`’ is False then ‘`exp3`’ is evaluated and value of ‘`exp3`’ becomes value of entire (complete) expression.

e.g. `int m, n;`
`m = 10;`
`n = 20;`
`z = (m < n) ? m : n ;`

In above example, ‘`z`’ takes the value of ‘`m`’ because ‘`exp1`’ (`m < n`) is True hence, value of ‘`z`’ is 10.

7) Bitwise operator:

Bitwise operators are used to manipulate the data at bit level. That is they are used to check the bits or shift them either left or right.

Also, bitwise operators are not operates over float and double type data. Following table shows bitwise operators with their meaning:

Operator	Meaning
&	Bitwise AND
	Bitwise OR
^	Bitwise exclusive OR
<<	Shift left
>>	Shift Right

The use of bitwise operator & , | and ^ is shown in following table:

Op1	Op2	Op1 & Op2	Op1 Op2	Op1 ^ Op2
1	1	1	1	0
1	0	0	1	1
0	1	0	1	1
0	0	0	0	0

• sizeof operator:

The sizeof operator is compile time operator which is used to find the total number of byte that the operand occupies.

The operand may be data type, variable or constant.

Use of sizeof operator:

It is used to find memory in byte that the operand occupies.

It is used to determine the length of array and structure when their sizes are not known in advance to the programmer.

Syntax:

`sizeof(operand) ;`

e.g.

1)
`sizeof(char);`

Here,
Operand is data type.

2)
`int n;`
`sizeof(n);`

Here,
Operand is variable.

3)
`sizeof(5.6);`

Here,
Operand is constant.

Note:

In 'C' language the by default floating number can be considered as double.
sizeof operator cannot operate on void data type.

e.g. Following program shows the use of sizeof operator:

```
#include< stdio.h >
#include< conio.h >
void main( )
```

```

{
    int    a,b,c,d;
    clrscr( );
    a = sizeof(char);
    b = sizeof(int);
    c = sizeof(float);
    d = sizeof(double);
    printf(" \n Character requires %d bytes",a);
    printf(" \n Integer requires %d bytes",b);
    printf(" \n Float requires %d bytes",c);
    printf(" \n Double requires %d bytes",d);
    getch( ) ;
}

```

Output:

```

Character requires 1bytes
Integer requires 2 bytes
Float requires 4 bytes
Double requires 8 bytes

```

3) Data Type:

Data: “Data is nothing but collection of raw information or unprocessed information that we provide for the computer for processing”

e.g. numbers, string, alphanumeric etc.

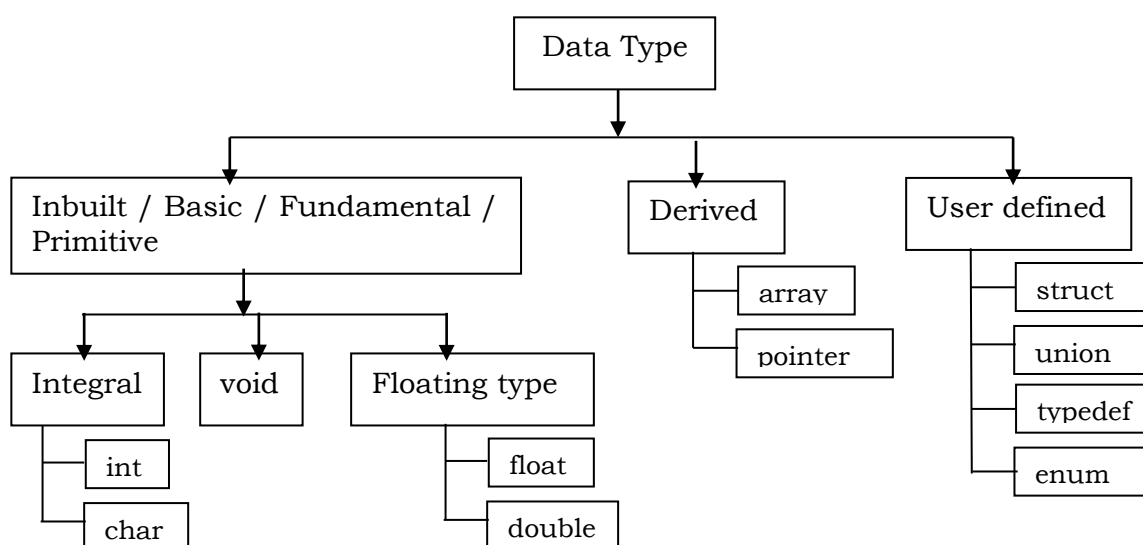
Data Type:

Concept: When we give data to the computer for processing at that time compiler does not know which type of input data is.

Generally, Data types are used to tell the compiler which type of input data is.

Definition: “Type of Data is called as Data Type”

Following figure shows data types in ‘C’ language:



The ‘C’ language supports for five fundamental or Basic or Inbuilt or primitive data types namely int, char, float, double and void.

Let’s see all these basic data types in details:

1) int data type: (Integer data type)

If we want to store integer value into variable then we have to declare the data type of that variable to int

Integers are whole numbers with the range value -32768 to 32767. Also to store one integer value 2 bytes (16 bits) of memory is required.

e.g. int x;

Here;

int data type is used to tell the compiler that 'x' is a variable and it only stores integer value.

2) char data type: (Character data type)

If we want to store single character into variable then we have to declare the data type of that variable to char

The character variable having the range value -128 to 127. Also to store one character 1 byte (8 bits) of memory is required.

e.g. char x;

Here;

char data type is used to tell the compiler that 'x' is a variable and it only stores single character.

3) float data type:

If we want to store floating value (the value having decimal point) into variable then we have to declare the data type of that variable to float

Floating numbers are real numbers having decimal point with the range value 3.4e - 38 to 3.4 e + 38. Also to store one float value 4 bytes (32 bits) of memory is required.

e.g. float x;

Here;

float data type is used to tell the compiler that 'x' is a variable and it only stores floating value.

4) double data type:

If we want to store floating value into variable then we can also declare the data type of that variable to double

double numbers are also real numbers having decimal point with the range value 1.7e - 308 to 1.7 e + 308. Also to store one double value 8 bytes (64 bits) of memory is required.

e.g. double x;

Here;

double data type is used to tell the compiler that 'x' is a variable and it only stores one double type value.

5) void data type:

The void is also basic data type in 'C' language. The void means no value. Generally void data type is used to specify return type of function. If we specify return type of function is void then that function does not return any value.

4) Identifier:

"Identifier is the name given by the user or programmer to the variable or pointer or array or any function"

While giving name we have to follow some rules and they are as follows (Rules to declare Identifier):

- 1) Identifier should not be keyword.
- 2) Identifier must begin with alphabets or underscore (_)
- 3) Identifier should not start with digit.
- 4) Identifier does not contain white space characters such as horizontal tab, vertical tab, space or new line etc.
- 5) Identifier contains the combination of alphabets (Small or Capital) and digits.

5) Variable:

“Variable is the name given to the memory location where the data is stored such quantity is called as Variable”

OR

“The quantity that changes during program execution is called as Variable”

Concept: The main concept behind variable is that every variable has an ability to store the data.

Syntax to declare variable:

DataType variableName ;

Here;

DataType is any valid data type in ‘C’ language.
 variableName is an identifier.

- e.g.
- 1) int x;
Here, ‘x’ is variable which holds only one integer value.
 - 2) float z;
Here, ‘z’ is variable which holds only one float value.

Rules to declare Variable:

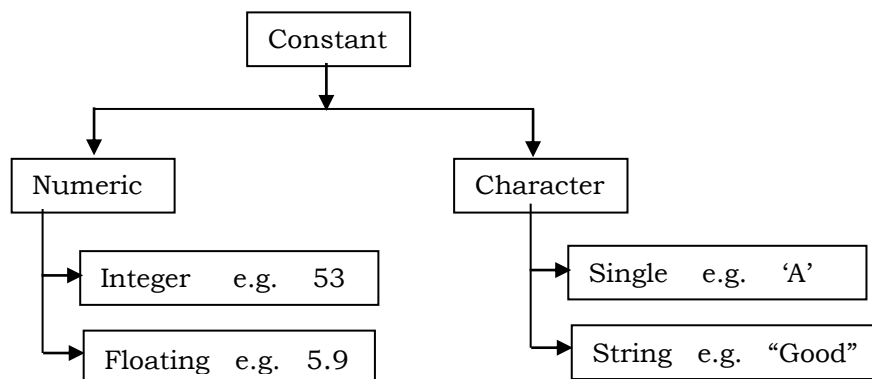
- 1) Variable should not be keyword.
- 2) Variable must begin with alphabets or underscore (_)
- 3) Variable should not start with digit.
- 4) Variable does not contain white space characters such as horizontal tab, vertical tab, space or new line etc.
- 5) Variable contains the combination of alphabets (Small or Capital) and digits.
- 6) Every variable must be declared before the use.

7) Constant:

“The quantity that does not change during the program execution such quantity is called as Constant”.

It is opposite of variable, also it has an ability to store the data.

‘C’ language supports several types of constants which are shown as fallows:



In ‘C’ language we can declare constant by two ways:

- 1) By using preprocessor directive #define

Syntax:

#define SymbolicName value

e.g.

#define PI 3.14

- 2) By using ‘const’ keyword:

Syntax:

Datatype const ConstantName = value;

e.g.

float const PI = 3.14;

Input and Output Operations:

Input and output operations are important to solve any problem.

Input means getting the values or data for processing and output means displaying the result on to the console.

In 'C' language, input and output operations are done with the help of inbuilt functions `scanf( )` and `printf( )` respectively.

scanf() :

`scanf( )` is inbuilt function which is used to take the input form keyboard for processing i.e. `scanf( )` is input function it means scan formatted. This function belongs to stdio.h header file.

Syntax to use scanf() :

```
scanf(" control string ", & arg1, & arg2 , . . . . . , & argN);
```

Here, control string only contains format code or format specifier.

`arg1, arg2, . . . . . , argN` are the variables along with address operator (&).

e.g.

```
1) int    x, y;
   scanf("%d%d", &x, &y);
```

Here, `%d` is format code which is used for single integer value

```
2) float  p, q;
   scanf("%f%f", &p, &q);
```

Here, `%f` is format code which is used for single float value

printf() :

`printf( )` is inbuilt function which is used to display the result or value over the console. i.e. `printf( )` is output function it means print formatted. Also, this function belongs to stdio.h header file.

Syntax to use printf() :

```
printf(" control string ", arg1, arg2 , . . . . . , argN);
```

The control string of `printf( )` function contains:

- 1) The characters that will be printed onto the screen as they appear.
- 2) Format code or Format specifier.
- 3) Back slash character constants such as new line, horizontal tab etc.

Also, `arg1, arg2, . . . . . , argN` are the variables.

e.g. 1)

```
printf("Hello");
```

OUTPUT:

Hello

2)

```
float  z = 53.42 ;
printf("Value=%f", z);
```

OUTPUT:

Value=53.42

3)

```
printf("Sachin \n Tendulkar");
OUTPUT:
```

Sachin
Tendulkar

Format Code or Format Specifier:

In 'C' language format code or format specifier are used in scanf () and printf () function corresponding to the variable's data type. The different format codes with their meaning are shown in following table:

Format Code	Meaning
%c	For single character
%d	For single integer
%e	For single float
%f	
%g	
%s	For single String
%lf	For single Double
%ld	For single long integer

Back slash character constant (Escape Sequence Characters):

In 'C' language some back slash character constants are used in only printf() function and the are as follows:

Back Slash Characters	Meaning
\n	New line
\t	Horizontal tab
\a	Audio beep
\b	Back space
\v	Vertical tab
\0	Null character

Reading and writing single character:

In 'C' language we can read and write single character by two ways:

I) By using scanf() & printf()function:

We can read single character by using scanf() as follows:

```
char    z;
scanf("%c", &z);
```

We can write single character by using printf() as follows:

```
char    z= 'A';
printf("%c", z);
```

II) By using getchar() & putchar()function:**getchar() :**

getchar() is inbuilt function which is used to read only single character from keyboard. That is getchar() is input function and is defined under stdio.h header file.

Syntax:

```
Char_type_variable= getchar( );
```

```
e.g:    char    z;
        z=getchar( );
```

here, getchar() function reads the single character and stores that character into 'z' variable.

putchar() :

putchar() is inbuilt function which is used to display only single character over the console. That is putchar() is output function and is defined under stdio.h header file.

Syntax: putchar(character);

```
e.g:    char    z= 'A';
        putchar(z);
```

here, putchar() function displays the single character 'A' stored in variable 'z'.